

Python Dasar

...

Eko Kurniawan Khannedy

Eko Kurniawan Khannedy

- Technical architect at one of the biggest ecommerce company in Indonesia
- 15+ years experiences
- www.programmerzamannow.com
- youtube.com/c/ProgrammerZamanNow



Eko Kurniawan Khannedy

- Telegram : [@khannedy](#)
- Linkedin : <https://www.linkedin.com/company/programmer-zaman-now/>
- Facebook : fb.com/ProgrammerZamanNow
- Instagram : instagram.com/programmerzamannow
- Youtube : youtube.com/c/ProgrammerZamanNow
- Telegram Channel : t.me/ProgrammerZamanNow
- Tiktok : <https://tiktok.com/@programmerzamannow>
- Email : echo.khannedy@gmail.com

Sebelum Belajar

- Mengerti Cara Menginstall Aplikasi di Sistem Operasi
- Mengerti Cara Menggunakan Terminal / Shell / Command Prompt
- Tahu apa itu Bahasa Pemrograman

Pengenalan Python

Pengenalan Python

- Python adalah bahasa pemrograman tingkat tinggi yang dibuat oleh Guido van Rossum dan pertama kali dirilis pada tahun 1991. Python terkenal karena sintaksnya yang sederhana dan mudah dipahami, mirip dengan bahasa Inggris.
- <https://www.python.org/>

Karakteristik Python

- Mudah dipelajari: Sintaks yang bersih dan sederhana
- Interpreted language: Tidak perlu dikompilasi terlebih dahulu
- Cross-platform: Berjalan di Windows, Mac, Linux
- Open source: Gratis dan bebas digunakan
- Versatile: Bisa digunakan untuk berbagai keperluan

Kegunaan Python

- Web Development: Django, Flask
- Data Science & Analytics: Pandas, NumPy, Matplotlib
- Machine Learning: TensorFlow, scikit-learn
- Automation/Scripting: Otomatisasi tugas-tugas repetitif
- Desktop Applications: Tkinter, PyQt
- Game Development: Pygame

Kelebihan Python

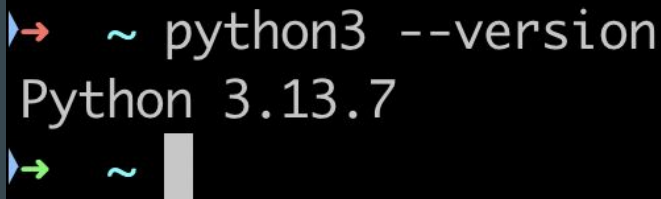
- Sintaks yang mudah dipahami
- Library yang sangat banyak
- Komunitas yang besar dan aktif
- Pengembangan yang cepat
- Multiplatform (bisa berjalan di banyak sistem operasi)

Kekurangan Python

- Kecepatan eksekusi lebih lambat dibanding bahasa compiled (C, C++, Java, Golang, dan lain-lain)
- Konsumsi memori yang lebih besar
- Tidak ideal untuk mobile development

Instalasi Python

- <https://www.python.org/downloads/>
- Gunakan terminal / shell / command prompt, ketik :
`python --version`
- Atau :
`python3 --version`



```
➤ ~ python3 --version
Python 3.13.7
➤ ~
```

Memilih dan Setup IDE/Text Editor

- Visual Studio Code : <https://code.visualstudio.com/>
- Pycharm : <https://www.jetbrains.com/pycharm/>

Program Hello Python

Python Interactive Shell

- Buka terminal dan ketik python atau python3
- Untuk membuat program yang menampilkan tulisan, kita bisa gunakan perintah `print()`, atau disebut sebagai function
- Di dalam kurung buka dan kurung tutup, kita bisa menambahkan tulisan, yang harus diawali petik dua dan diakhiri dengan petik dua, atau disebut String, misal : `print("Hello Python")`
- Untuk keluar dari Python Interactive Shell, kita bisa gunakan kata `quit`

Hello Python Interactive Shell

```
➤ ~ python3
```

```
Python 3.13.7 (main, Aug 14 2025, 11:12:11) [Clang 17.0.0 (clang-1700.0.13.3)] on darwin  
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> print("Hello Python")
```

```
Hello Python
```

```
>>> print("Hello Programmer Zaman Now")
```

```
Hello Programmer Zaman Now
```

```
>>> █
```

File Python

- Pada kenyataannya, saat membuat program menggunakan Python, kita biasanya tidak mungkin jarang menggunakan Python Interactive Shell, biasanya kita akan simpan kode program Python kita dalam file
- File Python menggunakan ekstensi / akhiran .py, menggunakan huruf kecil semua, jika butuh lebih dari satu kata, pisahkan kata dengan underscore (_), dan terakhir hindari spasi dan karakter khusus pada nama file

hello.py

 hello.py ×

```
1 print("Hello Python")
```

```
2
```

```
3 print("Hello Programmer Zaman Now")
```

```
4
```

```
5 |
```

Menjalankan File Python

- Untuk menjalankan kode program yang sudah kita buat, kita bisa menggunakan perintah :

```
python3 nama_file.py
```

- Pastikan kita mengetikkan perintah python3 di lokasi folder yang terdapat nama_file.py tersebut

Hasil Menjalankan File Python

```
→ belajar-python-dasar python3 hello.py
```

```
Hello Python
```

```
Hello Programmer Zaman Now
```

```
→ belajar-python-dasar
```

Komentar

- Saat membuat kode program Python, kadang kita butuh menambahkan komentar pada kode ini
- Ini biasanya digunakan untuk catatan agar kita ingat apa yang dilakukan oleh kode ini
- Komentar merupakan kode yang tidak akan dieksekusi oleh Python, jadi ini hanya digunakan sebagai catatan kita
- Kita bisa menggunakan tanda `#` di awal baris, yang artinya baris tersebut akan dianggap sebagai komentar

Komentar di hello.py

 hello.py ×

```
1 # program hello python
2 print("Hello Python")
3
4 # Ini adalah komentar
5 print("Hello Programmer Zaman Now")
6
7
```

Variabel dan Tipe Data

Apa itu Variabel?

- Variabel adalah "wadah" atau "kotak" untuk menyimpan data di dalam memori komputer. Dalam Python, variabel bersifat dinamis - tipe datanya bisa berubah selama program berjalan.
- Analogi Sederhana, bayangkan variabel seperti kotak dengan label. Kita bisa memasukkan berbagai jenis barang ke dalam kotak tersebut, dan labelnya adalah nama variabel.

variabel.py



variabel.py ×

```
1  # Membuat variabel seperti memberi label pada kotak
2  nama = "Budi"           # Kotak bernama 'nama' berisi text "Budi"
3  umur = 25               # Kotak bernama 'umur' berisi angka 25
4  tinggi = 170.5          # Kotak bernama 'tinggi' berisi angka desimal 170.5
5
6
```


Aturan Penamaan Variabel

- Tidak boleh dimulai dengan angka
- Tidak boleh dimulai dengan angka
- Tidak boleh pakai tanda minus
- Tidak boleh pakai keyword Python
- Tidak boleh ada spasi

Tipe Data Dasar Python

- Integer (int) - Bilangan Bulat
- Float - Bilangan Desimal
- String (str) - Teks
- Boolean (bool) - True/False

Tipe Data Integer

int.py ×

```
1  # Bilangan bulat positif dan negatif
2  umur = 25
3  tahun_lahir = 1998
4  saldo = -50000
5  nol = 0
6
7  # Bilangan besar (Python bisa handle angka sangat besar)
8  populasi_dunia = 7800000000
9  angka_besar = 123456789012345678901234567890
10
11 print(type(umur))           # <class 'int'>
12 print(type(angka_besar))    # <class 'int'>
13
```

Tipe Data Float

float.py ×

```
1  # Bilangan dengan koma
2  tinggi = 170.5
3  berat = 65.7
4  pi = 3.14159
5  suhu = -10.5
6
7  # Notasi scientific
8  speed_of_light = 3e8          # 3 x 10^8 = 300000000
9  very_small = 1e-6            # 0.000001
10
11 print(type(tinggi))           # <class 'float'>
12 print(speed_of_light)         # 300000000.0
13
```

Tipe Data String

```
str.py ×
1  # String dengan single quotes
2  nama = 'Budi Santoso'
3  kota = 'Jakarta'
4
5  # String dengan double quotes
6  alamat = "Jl. Merdeka No. 123"
7  pesan = "Selamat datang di Python!"
8
9  # String dengan triple quotes (multi-line)
10 puisi = """
11 Ini adalah puisi
12 Yang terdiri dari
13 Beberapa baris
14 """
15
16 # String kosong
17 empty_string = ""
18 another_empty = ''
19
20 print(type(nama))          # <class 'str'>
21
```

Tipe Data Boolean

```
bool.py ×  
1 # Boolean values  
2 is_student = True  
3 is_married = False  
4 has_license = True  
5  
6 print(type(is_student))      # <class 'bool'>  
7
```

Membuat Variabel

- Untuk membuat variabel, caranya seperti yang sudah kita lihat sebelumnya, yaitu menggunakan tanda = (sama dengan)
- Termasuk jika ingin mengubah data variabel yang sudah dibuat, kita juga bisa menggunakan tanda = (sama dengan)

variabel.py

variabel.py ×

```
1  # Assignment dasar
2  nama = "Alice"
3  umur = 30
4  tinggi = 165.0
5  is_active = True
6
7  # Multiple assignment
8  x = y = z = 0          # Semua variabel bernilai 0
9  a, b, c = 1, 2, 3      # a=1, b=2, c=3
10 name, age = "Bob", 25  # name="Bob", age=25
11
12 # Mengubah nilai variabel
13 skor = 80
14 print(skor)            # 80
15 skor = 95              # Mengubah nilai
16 print(skor)            # 95
17
```


Menggunakan Variabel

- Untuk menggunakan variabel, kita bisa langsung menyebut nama variabel nya
- Contoh misal jika kita ingin `print()` isi variabel, kita bisa langsung menggunakan function `print(nama_variabel)`
- Namun jika kita ingin menampilkan lebih dari satu data pada function `print()`, maka kita perlu menggunakan pemisah koma, misal :
`print(pertama, kedua, ketiga)`

menggunakan_variabel.py

```
menggunakan_variabel.py ×  
1  # Menggunakan variabel dalam operasi  
2  nama_depan = "John"  
3  nama_belakang = "Doe"  
4  nama_lengkap = nama_depan + " " + nama_belakang  
5  
6  panjang = 10  
7  lebar = 5  
8  luas = panjang * lebar  
9  
10 # Menggunakan variabel dalam print  
11 print("Nama lengkap:", nama_lengkap)  
12 print("Luas persegi panjang:", luas)  
13
```

Input dari Pengguna

- Kita bisa meminta input dari pengguna menggunakan function `input()`
- `input()` selalu menghasilkan string (teks).

input.py

 input.py ×

```
1  nama = input("Masukkan nama Anda: ")
2  print("Hello", nama)
3
4  umur_teks = input("Masukkan umur Anda: ")
5  print("Umur Anda:", umur_teks)
6  print("Tipe data:", type(umur_teks))
7
```

Konversi Tipe Data

- Konversi tipe data adalah mengubah data dari satu tipe ke tipe lainnya.
- Terdapat beberapa function yang bisa kita gunakan untuk melakukan konversi dari satu tipe data ke tipe data lain
- `int()`, mengubah string atau data lain menjadi integer (bilangan bulat)
- `str()`, mengubah integer atau data lain menjadi string (teks)
- `float()`, mengubah string atau integer menjadi float (bilangan desimal)
- `bool()`, mengubah data lain menjadi boolean (True/False)

konversi.py



konversi.py ×

```
1  nama = input("Masukkan nama Anda: ")
2  print("Hello", nama)
3
4  umur_teks = input("Masukkan umur Anda: ")
5  umur = int(umur_teks)
6  print("Umur Anda:", umur)
7  print("Tipe data:", type(umur))
```

8

String dan Manipulasi String

Apa itu String?

- String adalah tipe data untuk menyimpan teks atau kumpulan karakter. String ditulis dengan menggunakan tanda kutip, baik kutip tunggal (') atau kutip ganda (")

Menggabungkan String

- Untuk menggabungkan string, kita bisa menggunakan tanda + (tambah)
- Dan tipe data string tidak bisa digabungkan dengan tipe data lain seperti integer, float atau boolean, kita harus konversi dulu tipe data lain menjadi tipe data string

manipulasi_string.py



manipulasi_string.py ×

```
1  nama = "Budi"  
2  umur = 25  
3  
4  # Cara yang salah (akan error)  
5  # pesan = "Nama saya " + nama + ", umur " + umur  
6  
7  # Cara yang benar  
8  pesan = "Nama saya " + nama + ", umur " + str(umur)  
9  print(pesan)  
10
```

Panjang String

- Gunakan `len()` untuk mengetahui panjang string

manipulasi_string.py

```
manipulasi_string.py ×  
1  nama = "Budi"  
2  umur = 25  
3  
4  # Cara yang salah (akan error)  
5  # pesan = "Nama saya " + nama + ", umur " + umur  
6  
7  # Cara yang benar  
8  pesan = "Nama saya " + nama + ", umur " + str(umur)  
9  print(pesan)  
10  
11 print(len(nama))  
12 print(len(pesan))  
13
```

Mengakses Karakter dalam String (Indexing)

- Setiap karakter dalam string memiliki posisi (index) yang dimulai dari 0
- Untuk mengakses tiap karakter di string, kita bisa gunakan kurung kotak dan didalam kurung kotak tersebut disebutkan posisi index nya
- Kita juga bisa menggunakan posisi index negatif jika ingin mulai dari karakter paling belakang

manipulasi_string.py

```
nama = "Python"
print(nama[0]) # P (karakter pertama)
print(nama[1]) # y (karakter kedua)
print(nama[2]) # t (karakter ketiga)

nama = "Python"
print(nama[-1]) # n (karakter terakhir)
print(nama[-2]) # o (karakter kedua dari belakang)
print(nama[-3]) # h (karakter ketiga dari belakang)
```

String Slicing

- String slicing digunakan untuk mengambil sebagian dari string
- Kita bisa menggunakan kurung kotak dimana di dalam kurung kotak terdapat 2 posisi index dipisahkan oleh : (titik dua), itu menunjukkan posisi awal dan akhir karakter yang ingin kita ambil
- Jika kita tidak sebutkan posisi index di awal, berarti akan menggunakan default (nilai bawaan) dengan 0
- Dan jika tidak menyebutkan posisi index di akhir, berarti akan menggunakan default (nilai bawaan) dengan posisi index terakhir

manipulasi_string.py

```
nama = "Python"
print(nama[0:3])    # Pyt (index 0, 1, 2)
print(nama[2:5])    # tho (index 2, 3, 4)
print(nama[1:4])    # yth (index 1, 2, 3)

nama = "Python"
print(nama[:3])     # Pyt (dari awal sampai index 2)
print(nama[2:])     # thon (dari index 2 sampai akhir)
print(nama[:])      # Python (seluruh string)
```


String Methods (Fungsi String)

- Tipe data string memiliki method (fungsi yang menempel pada tipe data), ada banyak sekali method yang bisa digunakan, seperti
- `upper()` untuk mengubah semua karakter string menjadi huruf besar semua
- `lower()` untuk mengubah semua karakter string menjadi huruf kecil semua
- `title()` untuk mengubah setiap awal karakter di kata menjadi huruf besar
- `capitalize()` untuk mengubah awal karakter menjadi huruf besar
- `strip()` untuk menghilangkan karakter kosong (seperti spasi) di awal dan akhir
- `replace(dari, menjadi)` untuk mengganti bagian tertentu dalam string
- `count(text)` untuk menghitung berapa kali text muncul
- `find(text)` untuk mencari posisi text muncul

manipulasi_string.py

```
nama = "Alice"
nama_upper = nama.upper()
print(nama_upper) # ALICE
nama_lower = nama.lower()
print(nama_lower) # alice

nama = "john doe"
nama_title = nama.title()
print(nama.title()) # John Doe
nama_capitalize = nama.capitalize()
print(nama.capitalize()) # John doe
```

```
nama = "  Alice  "
nama_strip = nama.strip()
print(nama_strip) # 'Alice'

kalimat = "I love Java"
kalimat_baru = kalimat.replace("Java", "Python")
print(kalimat_baru) # I love Python

nama = "Mississippi"
jumlah_s = nama.count("s")
print(jumlah_s) # 4

kalimat = "Python Programming"
posisi = kalimat.find("Programming")
print(posisi) # 7
```

Escape Characters

- Escape Characters merupakan karakter-karakter khusus dalam string, untuk menggunakan karakter khusus kita perlu menggunakan aturan tertentu
- `\n` (New Line)
- `\t` (Tab)
- `\` (Backslash)
- `"` dan `'` (Kutip dalam String), perlu menambahkan `\` diawal

manipulasi_string.py

```
kalimat = "Baris pertama\nBaris kedua"  
print(kalimat)  
  
data = "Nama:\tAlice\nUmur:\t25"  
print(data)  
  
path = "C:\\Users\\Alice\\Documents"  
print(path)  
  
kalimat1 = "Dia berkata \"Hello\" kepada saya"  
print(kalimat1)  
  
kalimat2 = 'I\'m learning Python'  
print(kalimat2)
```

String Interpolation

- String interpolation adalah cara modern untuk menggabungkan variabel dengan teks.
- Ini lebih mudah dibaca dan lebih efisien daripada concatenation menggunakan + (plus).
- f-strings adalah cara terbaru dan paling direkomendasikan untuk string interpolation di Python, yaitu dengan menambah huruf f di awal ketika membuat string
- Selanjutnya di dalam nilai string, kita bisa menggunakan kurung kurawal untuk mengakses variabel lain

manipulasi_string.py

```
nama = "Alice"
umur = 25
kota = "Jakarta"

# Menggunakan f-string
profil = f"Halo, nama saya {nama}, umur {umur} tahun, tinggal di {kota}"
print(profil)

# Lebih jelas dibandingkan concatenation
profil_lama = "Halo, nama saya " + nama + ", umur " + str(umur) + " tahun, tinggal di " + kota
print(profil_lama)
```

f-strings dengan Expressions

- f-strings dapat mengeksekusi ekspresi langsung di dalamnya

manipulasi_string.py

```
harga = 100000
jumlah = 3

# Operasi matematika dalam f-string
total = f"Total: Rp {harga * jumlah:,}"
print(total)  # Total: Rp 300,000

# Method calls dalam f-string
nama = "john doe"
salam = f"Halo {nama.upper()}!"
print(salam)  # Halo John Doe!
```


Operator

Apa itu Operator?

- Operator adalah simbol yang digunakan untuk melakukan operasi pada variabel dan nilai. Python memiliki beberapa jenis operator yang bisa kita gunakan untuk memanipulasi data.

Operator Aritmatika

- Operator aritmatika digunakan untuk melakukan operasi matematika.
- Terdapat beberapa jenis operator aritmatika
- Operator dasar, seperti + (tambah), - (kurang), * (kali), / (bagi)
- Operator pembagian bulat dan modulo, seperti // (pembagian bulat), % (modulo, sisa pembagian)
- Operator pangkat, seperti ** (pangkat)

operator.py

operator.py ×

```
1 a = 10
2 b = 3
3
4 print(a + b) # Penjumlahan: 13
5 print(a - b) # Pengurangan: 7
6 print(a * b) # Perkalian: 30
7 print(a / b) # Pembagian: 3.333...
8
9 print(a // b) # Pembagian bulat: 3 (hasil tanpa desimal)
10 print(a % b) # Modulo: 1 (siswa pembagian)
11
12 # Contoh praktis modulo
13 print(10 % 2) # 0 (10 habis dibagi 2)
14 print(11 % 2) # 1 (siswa 1 ketika 11 dibagi 2)
```

```
15
16 a = 2
17 b = 3
18
19 print(a ** b) # Pangkat: 8 (2 pangkat 3)
20
21 # Contoh lain
22 print(5 ** 2) # 25 (5 pangkat 2)
23 print(3 ** 4) # 81 (3 pangkat 4)
```

Operator Assignment (Penugasan)

- Operator assignment digunakan untuk memberikan nilai pada variabel, seperti yang sudah pernah kita bahas, menggunakan tanda = (sama dengan)
- Compound Assignment, yaitu operator yang menggabungkan operasi aritmatika dengan assignment

operator.py

```
x = 10
print("x awal:", x)

x += 5 # Sama dengan: x = x + 5
print("x setelah += 5:", x) # 15

x -= 3 # Sama dengan: x = x - 3
print("x setelah -= 3:", x) # 12

x *= 2 # Sama dengan: x = x * 2
print("x setelah *= 2:", x) # 24

x /= 4 # Sama dengan: x = x / 4
print("x setelah /= 4:", x) # 6.0
```

Operator Perbandingan

- Operator perbandingan digunakan untuk membandingkan dua nilai. Hasil dari operasi ini adalah True atau False.

operator.py

```
print(a > b)      # Lebih besar: True
print(a < b)      # Lebih kecil: False
print(a >= b)     # Lebih besar atau sama: True
print(a <= b)     # Lebih kecil atau sama: False
print(a == b)     # Sama dengan: False
print(a != b)     # Tidak sama dengan: True
```

```
nama1 = "Alice"
nama2 = "Bob"
nama3 = "Alice"
```

```
print(nama1 == nama3)  # True
print(nama1 == nama2)  # False
print(nama1 != nama2)  # True
```


Operator Logika

- Operator logika digunakan untuk menggabungkan atau memodifikasi nilai boolean.
- and (dan), menghasilkan True jika kedua kondisi True
- or (atau), menghasilkan True jika salah satu kondisi True
- not (tidak), membalik nilai boolean

operator.py

```
umur = 25
print(umur > 18 and umur < 30)  # True (25 > 18 dan 25 < 30)

hari = "Sabtu"
print(hari == "Sabtu" or hari == "Minggu")  # True

aktif = True
print(not aktif)  # False
```

Operator String

- Operator khusus untuk bekerja dengan string.
- Concatenation (+), menambah string
- Repetition (*), mengulang string sejumlah angka
- Membership (in), mengecek apakah text ada dalam string

operator.py

```
nama_depan = "John"
nama_belakang = "Doe"
nama_lengkap = nama_depan + " " + nama_belakang
print(nama_lengkap)  # John Doe
```

```
kata = "Hello"
print(kata * 3)      # HelloHelloHello
```

```
garis = "-"
print(garis * 20)    # -----
```

```
kalimat = "Python adalah bahasa pemrograman"
print("Python" in kalimat)    # True
print("Java" in kalimat)     # False
print("adalah" in kalimat)   # True
```

Prioritas Operator

- Operator memiliki urutan prioritas (precedence). Operator dengan prioritas lebih tinggi akan dieksekusi terlebih dahulu
- Urutan Prioritas (dari tinggi ke rendah):
 - `**` (pangkat)
 - `*`, `/`, `//`, `%` (perkalian, pembagian)
 - `+`, `-` (penjumlahan, pengurangan)
 - `==`, `!=`, `<`, `>`, `<=`, `>=` (perbandingan)
 - `not` (logika not)
 - `and` (logika and)
 - `or` (logika or)

Kontrol Alur Program

Apa itu Kontrol Alur Program?

- Kontrol alur program adalah cara untuk mengatur jalannya program berdasarkan kondisi tertentu.
- Dengan kontrol alur, program bisa mengambil keputusan dan melakukan tindakan yang berbeda tergantung pada situasi.

Pernyataan If

- Pernyataan if digunakan untuk menjalankan kode hanya jika kondisi tertentu bernilai True.
- Setelah if, harus ada : (titik dua)
- Kode yang akan dijalankan jika kondisi True harus diindentasi (diberi spasi di awal)

if.py

if.py ×

```
1 angka = int(input("Masukkan angka: "))
```

```
2  
3 if angka > 0:
```

```
4     print("Angka positif")
```

```
5  
6 if angka < 0:
```

```
7     print("Angka negatif")
```

```
8  
9 if angka == 0:
```

```
10     print("Angka nol")  
11
```

Pernyataan if-else

- if-else digunakan ketika kita ingin menjalankan kode yang berbeda untuk kondisi True dan False.

if_else.py

 if_else.py ×

```
1  nilai = int(input("Masukkan nilai: "))
2
3  if nilai >= 60:
4      print("Anda lulus")
5  else:
6      print("Anda tidak lulus")
```

Pernyataan elif

- elif (else if) digunakan untuk mengecek beberapa kondisi secara berurutan.

elif.py

 elif.py ×

```
1  nilai = int(input("Masukkan nilai: "))
2
3  if nilai >= 90:
4      print("Grade A")
5  elif nilai >= 80:
6      print("Grade B")
7  elif nilai >= 70:
8      print("Grade C")
9  elif nilai >= 60:
10     print("Grade D")
11 else:
12     print("Grade E")
```

Kondisi dengan Operator Logika

- Karena kondisi harus bernilai boolean, artinya kita bisa menggunakan operator, misal operator logika, seperti and, or dan not

operator_logika.py

```
operator_logika.py ×  
1 umur = int(input("Masukkan umur: "))  
2 punya_sim = input("Punya SIM? (ya/tidak): ")  
3  
4 if umur >= 17 and punya_sim == "ya":  
5     print("Boleh mengendarai motor")  
6 else:  
7     print("Tidak boleh mengendarai motor")
```

Nested if (if Bersarang)

- Kita bisa menempatkan if di dalam if yang lain

nested_if.py

```
nested_if.py ×  
1 username = input("Username: ")  
2 password = input("Password: ")  
3  
4 if username == "admin":  
5     if password == "123456":  
6         print("Login berhasil")  
7         print("Selamat datang Admin")  
8     else:  
9         print("Password salah")  
10 else:  
11     print("Username tidak ditemukan")
```

Pernyataan Match Case

- match-case adalah fitur alternatif yang lebih bersih untuk elif yang banyak

match_case.py

```
match_case.py ×  
1 hari = input("Masukkan nama hari: ").lower()  
2  
3 match hari:  
4     case "senin" | "selasa" | "rabu" | "kamis" | "jumat":  
5         print("Hari kerja")  
6     case "sabtu" | "minggu":  
7         print("Hari libur")  
8     case _:  
9         print("Nama hari tidak valid")
```

Conditional Expression (Ternary Operator)

- Conditional expression memungkinkan kita menulis kondisi sederhana dalam satu baris

conditional_expression.py



conditional_expresion.py ×

```
1  angka = int(input("Masukkan angka: "))
2
3  # Dengan if-else biasa
4  if angka > 0:
5      hasil = "Positif"
6  else:
7      hasil = "Non-positif"
8
9  # Dengan ternary operator (lebih singkat)
10 hasil = "Positif" if angka > 0 else "Non-positif"
11 print("Angka tersebut:", hasil)
```

Perulangan

Apa itu Perulangan?

- Perulangan (loop) adalah cara untuk menjalankan kode yang sama berulang kali.
- Dengan perulangan, kita bisa menghemat waktu dan membuat kode lebih efisien daripada menulis kode yang sama berkali-kali.

For Loop

- For loop digunakan untuk mengulang kode sejumlah tertentu atau mengiterasi melalui sekumpulan data.
- For Loop biasanya menggunakan function `range()` untuk membuat urutan angka untuk perulangannya

for_range.py

for_range.py ×

```
1  # Mencetak angka 0 sampai 4
2  for i in range(5):
3      print(i)
4
5  # Mencetak angka 1 sampai 5
6  for i in range(1, 6):
7      print(i)
8
9  # Mencetak "Hello" sebanyak 3 kali
10 for i in range(3):
11     print("Hello")
12
13 # Menghitung mundur
14 print("Menghitung mundur:")
15 for i in range(5, 0, -1):
16     print(i)
```

For Loop dengan String

- For loop juga bisa digunakan untuk mengiterasi setiap karakter dalam string

for_string.py

 for_string.py ×

```
1  nama = "Python"
2  for huruf in nama:
3      print(huruf)
4
5  kata = input("Masukkan kata: ")
6  print("Huruf-huruf dalam kata:")
7  for karakter in kata:
8      print("-", karakter)
```

While Loop

- While loop mengulang kode selama kondisi tertentu masih True.

while_loop.py

```
while_loop.py ×  
1  # Mencetak angka 1 sampai 5  
2  angka = 1  
3  while angka <= 5:  
4      print(angka)  
5      angka += 1  
6  
7  # Input sampai benar  
8  password = ""  
9  while password != "123456":  
10     password = input("Masukkan password: ")  
11     if password != "123456":  
12         print("Password salah, coba lagi!")  
13  
14     print("Password benar!")
```

Break dan Continue

- break digunakan untuk keluar dari perulangan
- continue digunakan untuk melanjutkan ke iterasi berikutnya

break.py

break.py ×

```
1  # Game tebak angka dengan break
2  angka_rahasia = 7
3
4  while True:
5      tebakan = int(input("Tebak angka (1-10): "))
6
7      if tebakn == angka_rahasia:
8          print("Selamat! Anda benar!")
9          break
10     else:
11         print("Salah, coba lagi!")
```

continue.py



continue.py ×

```
1 # Mencetak angka ganjil saja
2 for i in range(10):
3     if i % 2 == 0: # Jika genap
4         continue # Lewati, lanjut ke angka berikutnya
5     💡 print(i) # Hanya mencetak angka ganjil
```


Loop dengan Else

- Python memiliki fitur unik dimana loop bisa memiliki clause else yang dieksekusi jika loop selesai secara normal (tidak dengan break).

for_else.py

```
for_else.py ×  
1  # Mencari huruf dalam kata  
2  kata = input("Masukkan kata: ")  
3  huruf_dicari = input("Masukkan huruf yang dicari: ")  
4  
5  for huruf in kata:  
6      if huruf == huruf_dicari:  
7          print("Huruf", huruf_dicari, "ditemukan dalam kata!")  
8          break  
9  else:  
10     print("Huruf", huruf_dicari, "tidak ditemukan dalam kata")
```

while_else.py

while_else.py ×

```
1  # Mencari password yang benar dengan batas percobaan ✓
2  password_benar = "python123"
3  percobaan = 0
4  max_percobaan = 3
5
6  while percobaan < max_percobaan:
7      password = input("Masukkan password: ")
8      percobaan += 1
9
10     if password == password_benar:
11         print("Login berhasil!")
12         break
13     else:
14         print("Password salah. Sisa percobaan:", max_percobaan - percobaan)
15 else:
16     print("Terlalu banyak percobaan gagal. Akses ditolak.")
```

Perulangan Bersarang (Nested Loop)

- Kita bisa menempatkan perulangan di dalam perulangan

nested_loop.py



nested_loop.py ×

```
1 # Tabel perkalian lengkap
2 print("Tabel Perkalian 1-5:")
3 for i in range(1, 6):
4     for j in range(1, 6):
5         hasil = i * j
6         print(i, "x", j, "=", hasil)
7 print("---")
```

Struktur Data

Apa itu Struktur Data?

- Data structures (struktur data) adalah cara untuk menyimpan dan mengorganisir data dalam program.
- Sejauh ini kita sudah belajar menyimpan satu nilai dalam satu variabel.
- Sekarang kita akan belajar cara menyimpan banyak nilai sekaligus.

List (Daftar)

- List adalah kumpulan data yang bisa menyimpan banyak nilai dalam satu variabel. List ditulis dengan menggunakan kurung siku `[]`.

list.py

list.py ×

```
1  # List kosong
2  daftar_kosong = []
3
4  # List dengan angka
5  angka = [1, 2, 3, 4, 5]
6  print(angka)
7
8  # List dengan string
9  nama = ["Alice", "Bob", "Charlie"]
10 print(nama)
11
12 # List campuran
13 campuran = ["Alice", 25, "Bob", 30]
14 print(campuran)
```

Manipulasi List

- Mengakses list, sama seperti string, list menggunakan index yang dimulai dari 0
- Mengubah Elemen List, bisa menggunakan index dan ubah seperti mengubah variabel
- Menambah Elemen ke List, bisa menggunakan method `append(data)` untuk menambah diakhir, dan `insert(index, data)` untuk menambah setelah index
- Menghapus Elemen dari List, bisa menggunakan method `remove(index)`, atau method `pop()` untuk menghapus elemen terakhir
- Untuk menghitung panjang list, kita bisa menggunakan function `len(list)`
- List bisa digabungkan dengan list lain menggunakan operator `+` (tambah)
- List juga sama seperti string, bisa di iterasi menggunakan for loop

list.py

```
# mengakses elemen
buah = ["apel", "jeruk", "pisang", "mangga"]
print(buah[0])    # apel
print(buah[1])    # jeruk
print(buah[2])    # pisang
print(buah[-1])   # mangga (elemen terakhir)

# mengubah elemen
warna = ["merah", "biru", "hijau"]
print(warna)
warna[1] = "kuning"
print(warna)      # ["merah", "kuning", "hijau"]
```

list.py

```
# append() - menambah di akhir
buah = ["apel", "jeruk"]
buah.append("pisang")
print(buah) # ["apel", "jeruk", "pisang"]
```

```
angka = [1, 2, 3, 4, 5]
```

```
# remove() - menghapus nilai tertentu
angka.remove(3)
print(angka) # [1, 2, 4, 5]
```

```
# pop() - menghapus elemen terakhir
angka.pop()
print(angka) # [1, 2, 4]
```

```
# del - menghapus berdasarkan index
del angka[0]
print(angka) # [2, 4]
```

```
# Panjang list
buah = ["apel", "jeruk", "pisang"]
print(len(buah)) # 3
```

```
# Menggabungkan list
list1 = [1, 2, 3]
list2 = [4, 5, 6]
gabungan = list1 + list2
print(gabungan) # [1, 2, 3, 4, 5, 6]
```

```
# Mengecek apakah elemen ada dalam list
if "apel" in buah:
    print("Apel tersedia")
```

```
buah = ["apel", "jeruk", "pisang"]
```

```
# Menggunakan for loop
print("Daftar buah:")
for item in buah:
    print("-", item)
```

```
# Dengan index
for i in range(len(buah)):
    print(str(i+1) + ".", buah[i])
```

Tuple

- Tuple mirip dengan list, tetapi tidak bisa diubah setelah dibuat. Tuple ditulis dengan kurung biasa ().
- Untuk membuat tuple, kita bisa buat langsung menggunakan kurung, misal (satu, dua, tiga), kita bisa buat data bebas berapapun yang kita mau
- Untuk mengakses data di tuple, sama seperti di list dan string, kita bisa gunakan index dengan kurung kotak
- Tuple tidak bisa diubah setelah dibuat, jadi tidak ada operasi untuk mengubah atau menghapus data di tuple
- Untuk mengetahui panjang tuple, kita bisa gunakan function `len(tuple)`
- Sama seperti list, kita juga bisa melakukan iterasi di tuple

tuple.py

 tuple.py ×

```
1 point = (5, 10)
2 print(point[0]) # 5
3 print(point[1]) # 10
4
5 # Untuk data yang tidak berubah
6 tanggal_lahir = (15, 8, 1990) # tanggal, bulan, tahun
7 print("Tanggal lahir:", tanggal_lahir)
8
9 # iterasi di tuple
10 for e in tanggal_lahir:
11     print(e)
12
13 # iterasi di tuple dengan index
14 for i in range(len(tanggal_lahir)):
15     print(tanggal_lahir[i])
16
```

Dictionary (Kamus)

- Dictionary menyimpan data dalam pasangan key-value (kunci-nilai). Dictionary ditulis dengan kurung kurawal {}.
- Untuk membuat data dictionary, kita perlu menentukan key dan juga value nya.
- Key di dictionary bisa tipe data seperti string, integer, float, boolean, namun biasanya kebanyakan orang menggunakan string untuk key nya
- Untuk mengakses dan mengubah value di dictionary, kita bisa gunakan key dengan kurung kotak, seperti di list
- Untuk menghapus value di dictionary, kita bisa gunakan kata kunci `del dict[key]`
- Untuk mengetahui panjang dictionary, kita bisa gunakan `len(dictionary)`
- Dictionary juga bisa di iterasi menggunakan perulangan `for` sama seperti list yang mengembalikan key nya

dictionary.py

dictionary.py ×

```
1 # Dictionary dengan data
2 siswa = {
3     "nama": "Alice",
4     "umur": 20,
5     "kelas": "12A"
6 }
7 print(siswa)
8
9 print(siswa["nama"]) # Alice
10 print(siswa["umur"]) # 20
11 print(siswa["kelas"]) # 12A
12
13 # Mengubah nilai
14 siswa["umur"] = 21
15 print(siswa)
```

```
16
17 # Menghapus key-value
18 del siswa["kelas"]
19 print(siswa)
20
21 # Mengiterasi keys
22 ∨ for key in siswa:
23     print(key, ":", siswa[key])
24
25 # Mengiterasi key-value pairs
26 ∨ for key, value in siswa.items():
27     💡 print(key, "=", value)
```


Set (Himpunan)

- Set adalah kumpulan data yang tidak berurutan dan tidak memiliki elemen duplikat.
- Set ditulis dengan kurung kurawal {} atau fungsi set().
- Untuk menambah data ke set, kita bisa gunakan method add(value)
- Untuk menghapus data di set, kita bisa gunakan method remove(value)
- Karena data di set tidak duplicate dan tidak berurutan, maka tidak bisa diakses menggunakan index
- Untuk mengakses data di set, biasanya menggunakan perulangan for

set.py

set.py ×

```
1  buah = {"apel", "jeruk", "pisang"}
2
3  # Menambah elemen
4  buah.add("mangga")
5  buah.add("mangga")
6  print(buah)
7
8  # Menghapus elemen
9  buah.remove("jeruk")
10 print(buah)
11
12 # Menampilkan elemen
13 for e in buah:
14     print(e)
```

Function

Apa itu Function?

- Function (fungsi) adalah blok kode yang dapat digunakan berulang kali.
- Function membantu kita menulis kode yang lebih terorganisir, mudah dibaca, dan mudah dipelihara.
- Dengan function, kita tidak perlu menulis kode yang sama berkali-kali.

Membuat Function

- Function dibuat menggunakan kata kunci `def`
- Lalu diikuti dengan nama function dan diakhiri dengan kurung buka dan tutup
- Biasanya nama function dibuat seperti nama variable, snake_case dan huruf kecil semua.
- Isi function harus terdapat spasi untuk menandakan bahwa itu adalah isi dari function nya
- Untuk memanggil function, kita bisa langsung panggil dengan nama_function diikuti dengan kurung buka tutup

function.py



function.py ×

```
1  def nama_function():
2      # Kode yang akan dijalankan
3      print("Hello dari function!")
4
5  # Memanggil function
6  nama_function()
7  nama_function()
8  nama_function()
```

Function dengan Parameter

- Parameter memungkinkan kita mengirim data ke function
- Caranya kita bisa tambahkan nama parameter diantara kurung buka dan kurung tutup di function
- Jika kita ingin menambahkan lebih dari satu parameter, kita bisa gunakan karakter koma sebagai pemisah parameter tersebut
- Saat memanggil function nya, maka kita wajib mengisi data pada parameter function nya

function.py

```
10  ✓ def sapa_nama(nama):  
11      print("Halo", nama)  
12      print("Senang bertemu dengan Anda!")  
13  
14      # Memanggil function dengan argument  
15      sapa_nama("Alice")  
16      sapa_nama("Bob")  
17  
18  ✓ def hitung_luas_persegi_panjang(panjang, lebar):  
19      luas = panjang * lebar  
20      print("Luas persegi panjang:", luas)  
21  
22      hitung_luas_persegi_panjang(10, 5)  
23      hitung_luas_persegi_panjang(8, 3)
```


Function dengan Return Value

- Saat membuat function, mungkin kita ingin function tersebut melakukan suatu kegiatan dan hasil kegiatan tersebut ingin kita dapatkan.
- Function bisa mengembalikan nilai dari kegiatan yang dilakukan di dalam function
- Function bisa mengembalikan nilai menggunakan kata kunci return
- Setelah kata kunci return, diikuti dengan nilai yang akan dikembalikan di function tersebut

function.py

```
25  ✓ def hitung_luas_lingkaran(radius):  
26      pi = 3.14159  
27      luas = pi * radius * radius  
28      return luas  
29  
30  luas1 = hitung_luas_lingkaran(5)  
31  luas2 = hitung_luas_lingkaran(10)  
32  
33  print("Luas lingkaran radius 5:", luas1)  
34  print("Luas lingkaran radius 10:", luas2)  
35  print("Total luas:", luas1 + luas2)
```

Default Parameter

- Saat kita menambahkan parameter di function, maka kita wajib mengisi nilai parameter tersebut ketika memanggil function nya
- Namun, kita juga bisa menambahkan default parameter, yaitu nilai awal untuk parameter tersebut
- Hal ini menjadikan jika kita tidak mengisi nilai pada parameter tersebut, maka parameter tersebut akan menggunakan nilai awal
- Posisi default parameter, tidak boleh di depan parameter biasa, harus di belakang parameter biasa
- Untuk membuat default parameter, cukup tambahkan = (sama dengan) dan diikuti dengan nilai awalnya

function.py

```
36
37  ✓ def sapa(nama, sapaan="Halo"):
38      print(sapaan, nama)
39
40      sapa("Alice")           # Menggunakan default "Halo"
41      sapa("Bob", "Hi")       # Menggunakan "Hi"
42      sapa("Charlie", "Hey")  # Menggunakan "Hey"
```

Keyword Argument

- Argument adalah nilai yang kita kirim ke parameter function ketika kita memanggil function
- Saat memanggil function, posisi argument harus sama dengan posisi parameter di function
- Namun, jika kita ingin mengubah posisi argument, sehingga posisinya tidak harus sama dengan posisi parameter, maka kita harus menyebutkan nama parameter nya saat memanggil function
- Fitur ini disebut Keyword Argument
- Saat menggunakan Keyword Argument, kita juga bisa mengkombinasikan dengan Argument biasa

function.py

```
def perkenalan(nama, umur, kota):  
    print("Nama:", nama)  
    print("Umur:", umur, "tahun")  
    print("Asal:", kota)  
    print("---")  
  
# Positional arguments (urutan harus sesuai)  
perkenalan("Alice", 25, "Jakarta")  
  
# Keyword arguments (urutan bebas)  
perkenalan(kota="Bandung", nama="Bob", umur=30)  
perkenalan(umur=28, kota="Surabaya", nama="Charlie")
```

function.py

```
▼ def buat_profil(nama, umur, kota="Jakarta", pekerjaan="Belum bekerja"):  
    print(f"=== PROFIL {nama.upper()} ===")  
    print(f"Umur: {umur} tahun")  
    print(f"Kota: {kota}")  
    print(f"Pekerjaan: {pekerjaan}")  
    print()  
  
# Positional + keyword arguments  
buat_profil("Alice", 25) # Menggunakan default  
buat_profil("Bob", 30, kota="Bandung")  
buat_profil("Charlie", 28, pekerjaan="Developer")  
buat_profil("Diana", 32, kota="Surabaya", pekerjaan="Designer")
```

Local Variable

- Saat kita membuat variabel di dalam function, maka variabel tersebut bersifat local di function itu, artinya tidak bisa digunakan di luar function

function.py

```
44  ✓ def fungsi_test():  
45      x = 10 # Local variable  
46      print("Nilai x di dalam fungsi:", x)  
47  
48  fungsi_test()  
49  # print(x) # Error! x tidak bisa diakses di luar fungsi
```

Global Variable

- Variable yang kita buat di luar function, maka bisa digunakan di dalam function, atau bersifat global
- Namun kita tidak bisa mengubah nilai dari global variable di dalam function, jika kita ingin mengubah nilai dari global variable, maka kita harus gunakan kata kunci global di dalam function nya

function.py

```
51  nama_global = "Alice" # Global variable
52
53  ✓ def tampilkan_nama():
54      print("Nama:", nama_global) # Bisa mengakses global variable
55
56  ✓ def ubah_nama():
57      global nama_global
58      nama_global = "Bob" # Mengubah global variable
59
60  tampilkan_nama() # Alice
61  ubah_nama()
62  tampilkan_nama() # Bob
```

Parameter Dinamis

- Python juga mendukung parameter dinamis
- Kita bisa menambahkan tanda * sebelum nama parameter untuk memberi tahu bahwa itu adalah parameter dinamis berupa List
- Atau kita bisa gunakan tanda ** sebelum nama parameter untuk memberi tahu bahwa itu adalah parameter berupa Dictionary

function.py

```
def cetak_list(*list):  
    for item in list:  
        print(item)
```

```
cetak_list(1, 2, 3, 4, 5)
```

```
def cetak_dict(**dict):  
    for key, value in dict.items():  
        print(f"{key}: {value}")
```

```
cetak_dict(nama="Alice", umur=25, kota="Jakarta")
```

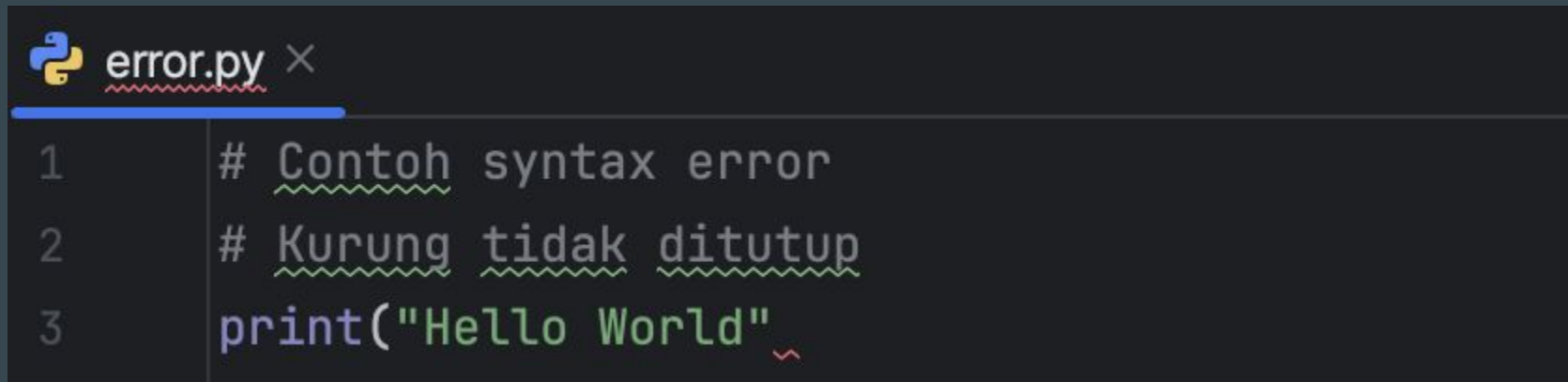
Penanganan Error

Apa itu Error?

- Error adalah kesalahan yang terjadi saat program berjalan.
- Ketika error terjadi, program akan berhenti dan menampilkan pesan error.

SyntaxError

- Kesalahan sintaks



```
error.py ×  
1 # Contoh syntax error  
2 # Kurung tidak ditutup  
3 print("Hello World")
```


NameError

- Variabel tidak ditemukan



error.py ×

```
1  # Contoh name error
2  # Variable 'nama' belum didefinisikan
3  print(nama)
```

TypeError

- Kesalahan tipe data



error.py ×

```
1  # Contoh type error yang akan menyebabkan error:  
2  # Tidak bisa menambah string dengan angka  
3  print("5" + 5)
```

ValueError

- Nilai tidak valid

 error.py ×

```
1 # Contoh value error yang akan menyebabkan error:  
2 # Tidak bisa convert "abc" ke integer  
3 angka = int("abc")  
4
```

IndexError

- Index tidak valid / tidak ditemukan

 error.py ×

```
1  # Contoh index error yang akan menyebabkan error:
2  list_data = [1, 2, 3]
3  # Index 5 tidak ada
4  print(list_data[5])
5
```

KeyError

- Key tidak valid / tidak ditemukan



error.py ×

```
1  # Contoh key error yang akan menyebabkan error:  
2  data = {"nama": "Alice"}  
3  # Key "umur" tidak ada  
4  print(data["umur"])
```

ZeroDivisionError

- Pembagian dengan nol

 error.py ×

```
1  # Contoh zero division error yang akan menyebabkan error:
2  # Tidak bisa dibagi dengan 0
3  print(10 / 0)
4
```

Penanganan Error

- Bagaimana untuk menangani error agar program kita tidak berhenti?
- Penanganan error memungkinkan program untuk menangani error dengan baik tanpa berhenti secara tiba-tiba.

Try-Except

- try-except digunakan untuk menangani error yang mungkin terjadi

error.py



error.py ×



```
1 print("== KALKULATOR SEDERHANA ==")
2
3 try:
4     angka1 = int(input("Angka pertama: "))
5     angka2 = int(input("Angka kedua: "))
6     hasil = angka1 / angka2
7     print("Hasil:", hasil)
8 except:
9     print("Terjadi error dalam perhitungan!")
10
11
```

⚠ 1 ✓ 9 ^ v

Menangani Error Spesifik

- Saat menangani error, mungkin kita ingin melakukan hal yang berbeda untuk jenis error yang berbeda
- Kita bisa menyebutkan tipe errornya setelah kata kunci `except`, sehingga jika terjadi error jenis tersebut, maka bagian `except` tersebut yang akan dieksekusi
- Kita bisa menambahkan lebih dari satu `except`

error.py



error.py ×



⚠ 1 ✓ 15 ^ v

```
1  try:
2      a = int(input("Masukkan angka pertama: "))
3      b = int(input("Masukkan angka kedua: "))
4      hasil = a / b
5      print("Hasil pembagian:", hasil)
6  except ValueError:
7      print("Mohon masukkan angka yang valid!")
8  except ZeroDivisionError:
9      print("Tidak bisa dibagi dengan nol!")
10 except:
11     print("Terjadi kesalahan yang lain!")
12
```

Try-Except-Else

- try-except juga memiliki bagian else seperti pada if dan for
- Bagian else dijalankan jika tidak ada error pada try-except
- Jika terjadi error, bagian else tidak akan dijalankan

error.py

```
13
14 try:
15     angka = int(input("Masukkan angka: "))
16 except ValueError:
17     print("Input tidak valid!")
18 else:
19     print("Angka yang Anda masukkan", angka)
20     if angka > 0:
21         print("Angka positif")
22     elif angka < 0:
23         print("Angka negatif")
24     else:
25         print("Angka nol")
26
```

Try Except Finally

- else di try-except hanya akan dijalankan ketika tidak ada error
- Bagaimana jika kita ingin melakukan sesuatu, baik itu jika terjadi error ataupun tidak?
- Kita bisa menggunakan finally
- finally selalu dijalankan, baik ada error atau tidak

error.py

```
26
27     try:
28         angka = int(input("Masukkan angka: "))
29         print("Angka Anda:", angka)
30     except ValueError:
31         print("Input tidak valid!")
32     finally:
33         print("Program selesai dijalankan")
```

File Input Output

Apa itu File I/O (Input/Output)?

- File I/O (Input/Output) adalah kemampuan program untuk membaca data dari file dan menulis data ke file.
- Dengan file I/O, kita bisa:
- Menyimpan data secara permanen
- Membaca data yang sudah tersimpan
- Membuat laporan dalam bentuk file
- Memproses data dari file eksternal
- Backup dan restore data aplikasi

Membuka dan Menutup File

- Untuk membuka file, kita bisa menggunakan function `open(file, mode)`
- Setelah selesai membuka file, jangan lupa tutup menggunakan function `close()` pada variabelnya
- Saat membuka file, kita harus tentukan mode nya

Mode File

Mode	Deskripsi	Contoh Penggunaan
r	Read (baca)	Membaca file yang sudah ada
w	Write (tuliskan)	Menulis file baru atau menimpa file lama
a	Append (tambah)	Menambah data di akhir file
x	Create (buat)	Membuat file baru, error jika sudah ada

Menulis ke File

- Untuk menulis ke file, kita bisa gunakan function `write(string)`
- Perlu diperhatikan, file yang sudah di `close()` tidak bisa ditulis lagi

file.py

file.py ×

```
1 print("=== SIMPAN DATA NILAI ===")
2
3 file = open("nilai_siswa.txt", "w")
4
5 while True:
6     nama = input("Nama siswa (enter untuk selesai): ")
7     if nama == "":
8         break
9
10    nilai = input("Nilai: ")
11
12    # Tulis ke file
13    file.write(nama + "," + nilai + "\n")
14    print("Data", nama, "berhasil disimpan!")
15
16 file.close()
17 print("Semua data berhasil disimpan ke nilai_siswa.txt")
```

✓ 16 ^ v

Membaca dari File

- Untuk membaca dari file, kita bisa menggunakan function `read()`, function ini akan mengembalikan seluruh isi file
- Namun mungkin kadang kita hanya ingin membaca file per-baris, maka kita bisa menggunakan iterasi `for` untuk membaca per baris

file.py

```
20 print("=== MENAMPILKAN DATA NILAI ===")
21
22 file = open("nilai_siswa.txt", "r")
23
24 for line in file:
25     data = line.strip().split(",")
26     print(data[0], ":", data[1])
27
28 file.close()
29
30 print("=== SELESAI ===")
```

With Statement (Rekomendasi)

- Saat membaca/menulis file, kita harus berhati-hati karena jika terjadi error, dan kita lupa melakukan `close()`, maka aplikasi kita bisa terjadi Memory Leak, yaitu kondisi dimana informasi file masih ada di memori aplikasi, padahal kita sudah tidak menggunakan file tersebut
- Hal ini sangat berbahaya karena penggunaan memori aplikasi kita akan semakin besar tanpa kita sadari
- `with` statement memastikan file otomatis tertutup, bahkan jika terjadi error
- Dengan menggunakan `with` statement, kita tidak perlu khawatir kita lupa menutup file menggunakan `close()`

file.py

```
19
20 print("=== MENAMPILKAN DATA NILAI ===")
21
22  ✓ with open("nilai_siswa.txt", "r") as file:
23  ✓     for line in file:
24         data = line.strip().split(",")
25         print(data[0], ":", data[1])
26
27 print("=== SELESAI ===")
28
```

Error Handling untuk File

- Walaupun selalu menutup file menggunakan with statement, tapi jika terjadi error, maka tetap aplikasi akan terhenti
- Oleh karena itu, kita tetap perlu menggunakan try-except
- Misal, bagaimana jika file yang kita baca itu tidak ada, maka akan terjadi error FileNotFoundError misalnya

file.py

```
19
20 print("=== MENAMPILKAN DATA NILAI ===")
21
22 try:
23     with open("nilai_siswa.txt", "r") as file:
24         for line in file:
25             data = line.strip().split(",")
26             print(data[0], ":", data[1])
27 except FileNotFoundError:
28     print("File nilai_siswa.txt tidak ditemukan.")
29
30 print("=== SELESAI ===")
31
```

Aplikasi Kalkulator Sederhana

Kalkulator Sederhana

- Penjumlahan
- Pengurangan
- Perkalian
- Pembagian

Aplikasi Permainan Tebak Angka

Permainan Tebak Angka

- Buat angka random
- Hint "terlalu besar" atau "terlalu kecil"
- Hitung jumlah tebakan
- Ada maksimal tebakan

Aplikasi Ujian Sekolah

Aplikasi Ujian Sekolah

- Membaca soal dari File
- Mengacak soal ujian
- Mengacak posisi jawaban soal
- Hasil score murid

Penutup